

Contract review

Source review of the Solidity contracts. Each item is the finding, then Richie Finance's resolution. Every item is informational. None of them take a deposit.

ORGANIZATION

`github.com/Richie-Finance`

REPOSITORY

`solidity-contracts`

COMMIT

`984795a`

REVIEWED

3 Nov 2026, 12:19am Sydney

MODEL

Grok 4.7, a frontier model

SITE

richie.finance

Result

This review was done with Grok 4.7, a frontier model, against Richie-Finance/solidity-contracts at `984795af484935442dddea842a30700858c971f8`. The organization is github.com/Richie-Finance.

Every finding is informational. None of them take a user's staked RICH, a farm or genesis deposit, or a user's CASH balance. New CASH minted by Treasury, and PDAI or RICH the DAO places in its own contracts, are protocol inventory. The pages after this index give the finding, then the resolution Richie Finance gave for it.

ID	RICHIE FINANCE RESOLUTION
H-1	DAO multisig exclusion is correct. Redeemed CASH is excluded because it is sent to that multisig. The Genesis address must be correct in source before Treasury is deployed.
H-2	Peg minting uses a 6-hour TWAP. A last trade does not set it. Holding a premium for that window, with no correcting trades, is unlikely on an active pool.
M-1	Catch-up of missed epochs is intentional. A keeper calls every six hours, and the staking page can call it too.
M-2	PDAI redemption is first come, first served, for whatever inventory is in the contract. CASH goes to the excluded multisig and can later be used for POL.
M-3	Rebate vesting stays at 3 days. Pricing is a multi-hour TWAP. The bondable RICH is the small DAO inventory, not user funds.
M-4	The harvest fee uses a 1-hour RICH/WPLS TWAP, kept fresh by a keeper. A bad print changes the PLS fee. It does not take the LP.
L-1	Boardroom resets its lock on deposit, claim, and withdraw. Defaults are 4 epochs to withdraw and 2 to claim. The maximum is 12 epochs.
L-2	The withdrawal recipient check was removed so a tax token cannot freeze withdrawals. Tax tokens are not listed.
L-3	Peg decreases an infinite allowance after a pull. Core flows do not lose funds.
L-4	Setting the Genesis operator to zero would be a deliberate admin action.
A-1	Treasury parameter changes sit with the 2-of-3 DAO multisig and are announced. They do not seize deposits.
A-2	Rebate controls sit with the same multisig. emergencyWithdraw can take only RICH that no vesting schedule claims.
A-3	The PSM fee defaults to 30% and cannot exceed the documented 50%. PDAI in the redeem contract is DAO inventory.

H-1 · Circulating supply

FINDING

Treasury's expansion base is total CASH supply minus a hardcoded exclusion list. At this commit that list is the DAO multisig and one Genesis address. There is no function that writes `excludedFromTotalSupply`. Deploying Genesis first and Treasury later does not update the array. After Treasury is deployed, the list is fixed in the bytecode.

```
address[] public excludedFromTotalSupply = [
    address(0x6831f815963FfCe95521271b94164eb4C82e7621), // DAO multisig
    address(0xb4ABb8B0F27D1fF26eA39725D996613Fa1b43073)  // Genesis
];
```

The redemption address was removed in `984795a`. Redeemed CASH is transferred to the DAO multisig in the same transaction, and that balance is already excluded, so it stays out of the expansion base while it sits there. `PegRedeem` does not check that `pegRecipient` is this multisig. The deployment has to set it.

The Genesis literal is still `0xb4ABb8B0F27D1fF26eA39725D996613Fa1b43073`. If Treasury is deployed with that address and Genesis is somewhere else, the 40,176,000 CASH genesis inventory stays in the expansion base. Unclaimed CASH inside Boardroom is still counted. That compounds later mints slightly. It does not take deposits.

> RICHIE FINANCE
>
> The DAO multisig `0x6831…7621` is correct. Genesis will be updated in source because Treasury is deployed later. The redemption address was removed from the exclusion list because redeemed CASH goes to that already-excluded multisig. No user funds are taken. The condition that remains is the one the code allows: replace the Genesis literal, recompile, and deploy Treasury from that bytecode. A later transaction cannot change the list.

H-2 · Peg TWAP

FINDING

`Oracle.update` rejects a sample shorter than its period. The peg deployment uses 6 hours. A swap writes the pair's cumulative price from the reserves before that swap, and `currentCumulativePrices` adds the current reserves only for seconds after the pair's last update. A trade in the update block does not set the average. Treasury mints from that average only when it is strictly above 1.01, and the mint is capped by the expansion tier.

Moving the print means holding a premium in the reserves for a large part of those 6 hours while other trades do not correct it. A same-block skew is excluded. Swallowing a failed update keeps the previous full-window observation, and Treasury rejects observations older than 6 hours. The 1-hour oracle in the docs is the RICH/WPLS pair used by the farm and rebates, not this peg window. `setPegOracle` can still point at a shorter oracle. That sits with the operator, covered under A-1.

> RICHIE FINANCE
>
> The 6-hour TWAP minimum is enforced regardless of the live spot. A consistent premium across the full window, with no other trades, is very unlikely once the pool is active. It does not take deposits. Accepted as residual oracle risk.

M-1 · Overdue epochs

FINDING

`allocateSeigniorage` is public, and each call advances one epoch that is already due. Several addresses can still clear several already-due epochs in one block. Each caught-up epoch mints new CASH to the Boardroom stakers at that moment. It does not take a deposit.

> RICHIE FINANCE
>
> Catch-up of missed epochs is intentional. A keeper calls every 6 hours, and eligible users can trigger it from the staking page. A backlog of outstanding epochs is unlikely. The residual same-block catch-up is accepted. No user funds are taken.

M-2 · Redemption

FINDING

`redeemPEG` pays PDAI one-for-one for as much inventory as the contract holds, first come first served, with no peg check and no burn. CASH is sent to the DAO multisig in that transaction. Users who redeem choose the trade. The PDAI is inventory the DAO placed in the contract, not a user deposit.

> RICHIE FINANCE
>
> This is the intended reserve. PDAI is fed in and redeemed first come, first served, for whatever is there. CASH goes to the DAO multisig in the same transaction and is excluded from circulating supply. It is not burned, so it can later be used for POL or another protocol use. While it sits in the multisig it is uncirculating. No user funds are taken.

M-3 · Rebate quote

FINDING

`bond` still has no `minShareOut`. `bondPLS` does. A bonder is filled at the quote that exists when their transaction lands. The RICH and asset quotes are the same style of TWAP as the peg: a same-block spot trade does not replace the average. The 10,000 RICH community allocation vests over 150 days from `Share`. That figure is not an on-chain cap inside `RebateTreasury`. The exposure is whatever RICH the DAO actually transfers in.

A bad quote can overpay a new bond from that inventory. It does not take RICH already vesting to a user.

> RICHIE FINANCE
>
> Vesting stays at the 3-day default. It will not be set to 1 second. Pricing uses the same TWAP style as the peg, with a minimum of about 2 hours depending on volume and liquidity. The cost of holding that average off-market removes the profit, given the small amount of RICH added at any time. The DAO has a maximum of 10,000 RICH dedicated to rebates, vested over 150 days. No user funds are taken.

M-4 · Harvest fee

FINDING

The farm PSM prices RICH with the RICH/WPLS oracle. The documented period for that oracle is 1 hour, shorter than the peg window. The same cumulative rule applies: the premium has to remain in the reserves for a meaningful part of the hour. A bad print can make the PLS fee high or low. It does not take staked LP. Withdrawal of the LP does not charge the fee.

> RICHIE FINANCE
>
> The 1-hour RICH/WPLS premium still has to be held, and other trades are likely to arbitrage it. A keeper updates the oracle. No user funds are taken.

L-1 · Boardroom lock

FINDING

Deposit, claim, and a withdraw that pays rewards all reset `epochTimerStart`. A user who claims restarts both the withdraw lock and the claim lock. Defaults are 4 epochs to withdraw and 2 epochs to claim. In `984795a` both values are capped at 12 epochs, which is 72 hours at a 6-hour epoch. The revert strings still say "must not exceed 56 epochs" even though the check is `≤ 12`. The cap is 12. The string is stale.

> RICHIE FINANCE
>
> Resetting the lock on every deposit, claim, and withdraw is intentional. Defaults stay at 4 epochs to withdraw and 2 to claim. Changes are announced in advance and made only if needed. The maximum was reduced from 56 epochs to 12. No user funds are taken.

L-2 · Withdrawal recipient check

FINDING

Commit `b17bc6e` removed the check that the user received the full stake token on `withdraw` and `emergencyWithdraw`. The pool still requires its own balance to fall by the full amount. A token that taxes the sender, or that does not move the full amount out of the pool, still reverts. A token that taxes only the recipient no longer does.

> RICHIE FINANCE
>
> The recipient check was removed because it blocks withdrawals of a tax token and would freeze those positions. Tax tokens will not be listed in Genesis or the Farm. For the tokens that will be listed, the removed check is not a protocol issue. No user funds are taken.

L-3 · Peg infinite allowance

FINDING

`Peg.transferFrom` decreases a max allowance instead of leaving the `type(uint256).max` sentinel in place. A caller that approved the maximum once will see that allowance fall by the amount pulled. Core flows do not depend on the sentinel staying at the maximum.

> RICHIE FINANCE
>
> No loss of funds from the core protocol. Accepted.

L-4 · Zero genesis operator

FINDING

`GenesisRewardPool.setOperator` still allows `address(0)`. That would freeze later pool configuration, because only the operator can change it. Withdrawals would keep working.

> RICHIE FINANCE
>
> The operator would have to be set to zero on purpose. No user funds are taken. Accepted as an admin footgun.

A-1 · Treasury operator

FINDING

The operator can replace the peg oracle, edit expansion tiers, the ceiling, bootstrap (only before start), fund splits, and Boardroom lockups. None of these calls transfer a user's deposit. `984795a` lowered the on-chain caps:

- Boardroom withdraw and reward lockups: maximum 12 epochs.
- Bootstrap length: maximum 28 epochs, and only before `startTime`.
- DAO share of a mint: maximum 15%. Dev share: maximum 5%. Combined: maximum 20%.

Two caps were not lowered. `setMaxExpansionTiersEntry` still allows a tier of 1,000, which is 10% of supply per epoch. `setBootstrap` still allows a bootstrap expansion of 10% per epoch. Those are the maximums the operator can reach, not the defaults. Defaults are locks of 4 and 2 epochs, 28 bootstrap epochs at 2.5%, then expansion of 2%, 1%, 0.5%, and 0.25% as CASH supply grows.

> RICHIE FINANCE
>
> The oracle, Boardroom lockups, extra funds, bootstrap, and the peg price ceiling may be updated for protocol health, and changes are announced beforehand. The Treasury operator is the DAO 2-of-3 multisig. No user funds are taken.

A-2 · Rebate owner

FINDING

Rebate parameters, the per-epoch asset sweep, and PLS withdrawal stay with the owner. `setBondParameters` allows a discount up to 50% and any vesting period above zero. `setAsset` can set any positive multiplier. A hostile multiplier would overpay new bonds from unallocated inventory. It cannot pull RICH already inside `totalVested`.

`emergencyWithdraw` does not pay out vested shares, and it does not take the RICH owed to bonders. For the share token it requires the amount to be at most the contract balance minus `totalVested`:

```
function emergencyWithdraw(IERC20 token, uint256 amnt) external onlyOwner {
    if (address(token) == address(Share)) {
        uint256 shareBalance = Share.balanceOf(address(this));
        require(
            shareBalance ≥ totalVested && amnt ≤ shareBalance - totalVested,
            "RebateTreasury: Share reserved for vesting"
        );
    }
    token.safeTransfer(owner(), amnt);
}
```

`totalVested` is the RICH still owed on schedules, including time that has already passed but has not been transferred to the user yet. The owner can withdraw only RICH that no schedule claims: unused funding, or RICH sent by mistake. If the DAO transfers in 1,000 RICH and nobody has bonded, `totalVested` is 0 and the owner can take the 1,000 back. After a user bonds 100, the owner can take at most the other 900. The 100 stays until `claimRewards` or a new bond pays the user through `_claimVested`. The buyback path sends 100% of the bonded assets each epoch. That inventory is not a user's deposit.

> RICHIE FINANCE
>
> These controls are for protocol health, they stay with the DAO multisig, and changes are announced. Vesting will not be set to 1 second. The 50% discount ceiling is the design. Sweeping bonded assets in full is necessary so the DAO can use them immediately. `emergencyWithdraw` takes unused inventory above `totalVested`. It cannot take RICH owed to bonders. No user funds are taken.

A-3 · Farm fee and PDAI reserve

FINDING

The PSM fee defaults to 30% and the contract maximum is 50%. The fee is extra PLS on harvest. It is not a cut of the LP. PDAI held in `PegRedeem` can be sent to the owner. It is not collateral a user locked in that contract.

> RICHIE FINANCE
>
> The 50% ceiling is the documented maximum, the default is 30%, and changes are announced. PDAI in the redeem contract is DAO inventory for POL, revenue, and below-peg redemptions, as the docs describe, and it is sent to the DAO multisig. It is not user collateral.

Before Treasury is deployed

> THE EXCLUSION LIST CANNOT BE CHANGED LATER
>
> `excludedFromTotalSupply` has no setter. The source still names Genesis as `0xb4ABb8B0F27D1fF26eA39725D996613Fa1b43073`. Replace that literal with the deployed Genesis address, recompile, and deploy Treasury from that bytecode.

Set PegRedeem's recipient to the DAO multisig `0x6831f815963FfCe95521271b94164eb4C82e7621`. CASH sent there is already outside circulating supply. The redeem contract does not check this itself. The Boardroom lock revert text can be updated from "56 epochs" to "12 epochs" so a failed call reports the real cap.

Scope

Peg, Share, Treasury, Boardroom, the Solidity 0.6.12 TWAP oracle, the rebate composite oracle, Farm, Genesis, PegRedeem, Rebates, and the PulseX zaps, in Richie-Finance/solidity-contracts.

Reviewed with Grok 4.7, a frontier model. This was a source review. It was not a fuzz run, a formal proof, or a diff against live PulseChain bytecode. The same notes are in AUDIT.md. The organization is github.com/Richie-Finance.